

VLAN Hopping Attacks

Switch Spoofing & Double Tagging — Junior Pentester Study Guide

Theory + Hands-on Lab Steps + Detection & Mitigation + Exam Notes

WHAT'S INSIDE

- How VLAN segmentation works and why attackers target it
- Switch Spoofing (DTP trunk negotiation abuse) — theory + lab walkthrough
- VLAN Double Tagging (802.1Q) — theory + lab walkthrough
- Tool reference: Yersinia, Wireshark, Scapy & alternatives
- Blue-team side: detection & mitigation checklist
- Quick-reference cheat sheet + practice drill

Source: CompTIA PenTest+ style lab walkthrough, expanded with additional attacker/defender detail.

01 Why Segmentation — and Why Attackers Care

The defensive concept VLAN hopping is designed to defeat

Network segmentation with VLANs (Virtual LANs) is one of the most common and effective defensive controls in a corporate network. Instead of running one flat network where every device can talk to every other device, the switch logically splits ports into separate broadcast domains. A guest Wi-Fi jack and the finance department's desktops can sit on the same physical switch but belong to completely different VLANs, with zero visibility into each other's traffic.

From an attacker's perspective, this is a wall. If you compromise a low-value segment (guest network, a lobby Ethernet jack, a conference room drop) you are normally stuck there. **VLAN hopping** is the technique of breaking out of your assigned VLAN and reaching traffic or hosts on a VLAN you were never supposed to touch — without physically re-cabling anything.

WHY THIS MATTERS ON THE EXAM AND IN REAL ENGAGEMENTS

CompTIA PenTest+ treats VLAN hopping as a core network-based attack technique. In the real world it rarely works out-of-the-box against modern, hardened switches — but misconfigured or legacy gear is still common, especially in older manufacturing plants, retail chains, and acquired companies that never had their network hardened after a merger.

LAB SCENARIO USED IN THIS GUIDE

Element	Detail
Attacker host	Kali Linux VM, placed on VLAN 11 ("Guest" network) — e.g. plugged into a lobby jack or joined via guest Wi-Fi
Target VLAN	VLAN 22 ("Accounting") — two hosts holding sensitive financial data
Switch config	Default / out-of-the-box switch settings — no manual hardening applied
Attacker goal	Get visibility into, or a foothold on, VLAN 22 traffic while physically connected only to VLAN 11

This mirrors a realistic engagement: you land on a low-trust network segment during an on-site or wireless assessment, and the client wants to know whether that segment can reach anything more sensitive.

02 Attack 1 — Switch Spoofing (DTP Abuse)

Tricking a switch port into becoming a trunk

THEORY

Switch ports can operate in two relevant modes: **access mode** (belongs to exactly one VLAN, what an end-user PC connects to) and **trunk mode** (carries tagged traffic for *multiple* VLANs at once — normally used for switch-to-switch or switch-to-router links).

Cisco switches historically ran **DTP (Dynamic Trunking Protocol)** on every port by default, set to "**dynamic desirable**" or "**dynamic auto**". In this state, if the switch hears DTP negotiation frames on a port claiming to be a switch that wants a trunk, it will happily negotiate the link *up into trunk mode*. A trunk port carries traffic for every VLAN allowed on that trunk by default — which, unless it has been restricted, means **all VLANs configured on the switch**.

Switch spoofing abuses this: the attacker's machine sends crafted DTP frames pretending to be a neighboring switch. If the port auto-negotiates to trunk mode, the attacker's NIC is now sitting on a trunk link and can see (and potentially inject into) every VLAN carried on that trunk.

PREREQUISITE FOR THIS ATTACK TO WORK

The target switch port must still be running DTP in a negotiating mode (dynamic auto / dynamic desirable) and DTP itself must not be disabled. On any switch hardened per vendor best practice, all user-facing ports are hard-set to access mode with DTP switched off — which kills this attack outright. Expect this to fail on modern managed switches.

PRACTICAL — LAB WALKTHROUGH

1. Recon the interface

Identify which NIC on the Kali box is connected to the target switch port (e.g. `eth0`). If dual-homed, confirm the correct interface before touching anything.

2. Baseline capture with Wireshark

Start Wireshark (or `tcpdump`) on that interface first. You should immediately see normal switch control-plane chatter: STP (Spanning Tree) BPDUs and CDP (Cisco Discovery Protocol) advertisements. This confirms you have L2 visibility and gives you a 'before' baseline.

3. Launch Yersinia

Yersinia is a purpose-built L2 attack framework. Launch the GUI as root: `sudo yersinia -G`. (Command-line mode is also available via `yersinia dtp -attack 1`.)

4. Select the DTP attack

In Yersinia: **Launch attack** → **choose the DTP protocol tab** → **"enable trunking"**. Do NOT tick the denial-of-service option — that's a different, noisier attack with a different objective.

5. Confirm negotiation in Wireshark

Switch back to Wireshark. You should see DTP frames going out and, if vulnerable, the port renegotiating — followed by 802.1Q tagged frames from VLANs you were not on before.

6. Validate the win

Look for ARP or broadcast traffic tagged for the target VLAN (e.g. traffic to/from a 192.168.22.x range when you started on 192.168.11.x). Seeing that traffic proves you're now on a trunk carrying the target VLAN.

```
# Example command-line flow (Kali, as root)
sudo airmon-ng check kill # optional, if wireless adapter conflicts
sudo wireshark & # start capture on the wired interface first
# Launch Yersinia GUI
sudo yersinia -G
# -> DTP tab -> Launch attack -> "enable trunking"
# CLI alternative
sudo yersinia dtp -attack 1 # 1 = enable trunking (send DTP desirable)
```

WHAT ACTUALLY HAPPENED IN THE LAB RUN

In the walkthrough this guide is based on, the DTP trunk negotiation succeeded (visible in Wireshark), but no real cross-VLAN traffic appeared beyond routine STP frames — a realistic outcome. A successful trunk negotiation does not always hand you useful data immediately; you may need to actively generate or wait for traffic, or pivot to double tagging instead.

03 Attack 2 — VLAN Double Tagging

Smuggling a frame across a VLAN boundary with two 802.1Q tags

THEORY

802.1Q is the standard that lets a single Ethernet frame carry a VLAN tag so a trunk link knows which VLAN the frame belongs to. **Double tagging** abuses how some switches process these tags: the attacker crafts a frame with two stacked 802.1Q tags.

- The **outer tag** matches the attacker's own (native) VLAN — VLAN 11 in this lab.
- The **inner tag** matches the target VLAN the attacker wants to reach — VLAN 22.

When the frame hits the first switch, the switch sees the outer tag matches its native VLAN and — because native VLAN traffic is conventionally sent untagged on trunk links — **strips the outer tag and forwards the frame** without re-examining it closely. The inner tag (VLAN 22) is now exposed as if it were the frame's only tag, so the next switch in the path forwards it straight onto VLAN 22. The frame has effectively "hopped" from VLAN 11 to VLAN 22 by hiding inside a tag the first switch trusted implicitly.

CRITICAL LIMITATION — KNOW THIS COLD FOR THE EXAM

Double tagging is **one-way / blind**. Return traffic from the target VLAN does NOT get double-tagged back to the attacker, because the reply is generated by a normal host on VLAN 22 that only knows how to send single-tagged (or untagged) frames. This makes the attack useful for blind injection (e.g. spoofed traffic, DoS, one-way exfil triggers) but not for interactive two-way sessions. It also only works when the attacker's own VLAN matches the trunk's native VLAN — another config-dependent precondition.

PRACTICAL — LAB WALKTHROUGH

1. Return to Yersinia

Same tool, different protocol tab: **Launch attack** → **802.1Q tab**. Select the option described as sending *802.1Q double-encapsulated packets*.

2. Fire the attack

Confirm and launch. Yersinia builds and sends crafted double-tagged frames from the attacker interface.

3. Watch Wireshark on the way out

You'll see the crafted 802.1Q frames leave the attacker NIC — this confirms Yersinia is generating them correctly.

4. Watch for the target VLAN's address range

In this lab, VLAN 11 hosts sit on 192.168.11.0/24 and VLAN 22 on 192.168.22.0/24 (a very common real-world convention — VLAN ID often maps directly into the third IP octet). Watch for ARP or other broadcast traffic addressed to the 192.168.22.x range.

5. Confirm the hop

Seeing traffic destined for a host like 192.168.22.11 on your capture — while your NIC is only physically connected to VLAN 11 — confirms the double-tag frame successfully reached VLAN 22.

6. Decide next steps

From here, realistic follow-on actions include blind spoofed traffic injection, a denial-of-service attempt against a VLAN 22 host, or (with more setup) attempting a one-way man-in-the-middle / data exfiltration trigger. Full bidirectional MITM

is not possible with double tagging alone.

```
# Yersinia GUI path
sudo yersinia -G
# -> 802.1Q tab -> Launch attack -> "send 802.1Q double encapsulated packets"

# CLI alternative
sudo yersinia dot1q -attack 1

# Manual crafting alternative with Scapy (more control, no GUI)
sudo python3
>>> from scapy.all import *
>>> frame = Ether()/Dot1Q(vlan=11)/Dot1Q(vlan=22)/IP(dst='192.168.22.11')/ICMP()
>>> sendp(frame, iface='eth0')
```

Yersinia is the fastest way to demonstrate the attack in a lab or during a live exam-style walkthrough because both attacks are pre-built. In a real engagement, **Scapy** gives you far more control over the crafted frame (source MAC, inner payload, protocol) once you've proven the concept works.

04 Tool Reference

What was used in the lab, plus additional tools worth knowing

Tool	Role in this attack
Wireshark	Passive capture/validation. Confirms DTP negotiation, shows tagged frames, and proves cross-VLAN traffic reached your NIC. Always run it before and during any active attack.
Yersinia	Purpose-built L2 attack framework (DTP, STP, CDP, DHCP, HSRP, 802.1Q, and more). Fastest way to run switch spoofing and double tagging with minimal setup; GUI (-G) or CLI.
Scapy	Python packet-crafting library. Use once you understand the attack, to build custom double-tagged frames, spoof source addresses, or chain the hop into a follow-on payload.
tcpdump	Lightweight CLI alternative to Wireshark for headless boxes or when you need to pipe capture into a script (<code>tcpdump -i eth0 -w cap.pcap</code>).
macof (dsniff suite)	Not used in this lab, but commonly paired with VLAN attacks — floods a switch's CAM table to force it into fail-open hub mode, another route to see traffic outside your VLAN.
Frogger / VLANPWN / CDP tools	Additional open-source utilities for CDP-based recon and VLAN attacks; useful when Yersinia isn't available or you need CDP spoofing specifically to fingerprint neighboring switches first.
Nmap / arp-scan	Once you land on a new VLAN (via either technique), use these to enumerate what's actually reachable before going further.

RECON BEFORE YOU ATTACK

Before attempting either technique on an engagement, passively capture CDP/LLDP and STP traffic for a few minutes. CDP frames from Cisco gear will often tell you the switch model, IOS version, and neighboring port — useful for judging whether DTP is likely to still be enabled by default on that hardware.

05 Attacker Strategy & Methodology

How to actually approach this on an engagement, not just in the lab

A REPEATABLE APPROACH

- **Map your starting position.** Identify your VLAN, IP range, default gateway, and what DHCP handed you. Passive-sniff STP/CDP/LLDP for a baseline before sending anything.
- **Fingerprint the switch.** CDP/LLDP capture, MAC OUI lookups, and banner grabs (if you can reach management interfaces) tell you vendor and rough age of the hardware — older/unmanaged gear is far more likely to be DTP-vulnerable.
- **Try switch spoofing first.** It's cheap to test and, if it works, gives you full trunk visibility rather than one blind VLAN — strictly more valuable than double tagging.
- **Fall back to double tagging if spoofing fails.** Requires knowing (or guessing) the trunk's native VLAN ID — often VLAN 1 by default, or discoverable from captured DTP/STP frames.
- **Validate before you exploit.** Confirm the hop with passive capture (ARP/broadcast from the target VLAN) before attempting anything noisier like injection or DoS — noisy failed attempts burn your stealth budget.
- **Escalate carefully.** A successful hop is a beachhead, not a win. Follow with careful, rate-limited scanning (arp-scan/nmap -T2) of the newly-visible range — aggressive scanning here is a common way pentesters get caught by the blue team.
- **Document everything.** Screenshot the Wireshark captures showing before/after traffic, note exact Yersinia settings used, and record switch fingerprint info — this evidence is what makes the finding credible in the report.

DECIDING WHICH TECHNIQUE TO USE

Situation	Preferred technique
Port still runs DTP in auto/desirable mode	Switch spoofing — full trunk access, two-way traffic
DTP is off, but native VLAN is guessable/known	Double tagging — blind, one-way, still useful for injection/DoS
Both are hardened / patched	Pivot to other L2 attacks: CAM table flooding (macof), ARP spoofing within your own VLAN, rogue DHCP, or physical/wireless re-entry into a different segment
You need bidirectional access to the target VLAN	Switch spoofing only — double tagging cannot return traffic to you
Goal is just to prove segmentation is weak (client report)	Either — the finding is the same: "VLAN hopping possible from an untrusted segment"

06 Detection & Mitigation

The blue-team side — what a junior pentester should recommend in the report

Every offensive finding needs a matching, practical remediation in your report. For VLAN hopping, the fixes are well-established and cheap to implement — which also means an auditor/client will expect you to know them cold.

Control	What it does
Disable DTP on all access ports	Set every user-facing port to <code>switchport mode access</code> and <code>switchport nonegotiate</code> — kills switch spoofing outright.
Never use VLAN 1 as the native VLAN	Move the native VLAN to an unused ID and explicitly tag it, so double tagging can't rely on the default trunk assumption.
Prune unused VLANs off trunks	Explicitly allow only the VLANs a trunk actually needs (<code>switchport trunk allowed vlan</code>) instead of allowing all VLANs by default.
Disable unused switch ports / use port security	Shut down unused ports, and limit learned MAC addresses per port to prevent rogue devices from easily joining.
Private VLANs (PVLANS) / ACLs	Add a second layer of isolation between hosts even within the same VLAN, limiting blast radius if a hop does succeed.
DHCP snooping + Dynamic ARP Inspection	Doesn't stop VLAN hopping directly, but limits what an attacker can do once on a new segment (spoofed DHCP/ARP).
Monitor for DTP/CDP anomalies (NAC/IDS)	Alert on unexpected DTP negotiation attempts or CDP frames from non-switch MAC/OUI ranges — this is the actual detection signal for this attack.

REALITY CHECK

As the lab instructor notes, hardened / modern switch defaults already block a lot of this (DTP off by default on many newer platforms, native VLAN warnings, etc.). Don't expect these attacks to work against a well-maintained enterprise network — but legacy gear, SMB environments, OT/ICS networks, and post-acquisition IT are still frequently vulnerable.

07 Quick-Reference Cheat Sheet

One page to glance at before a lab or exam question

	Switch Spoofing	Double Tagging
Abuses	DTP trunk auto-negotiation	How switches handle stacked 802.1Q tags + native VLAN
Requires	Port left in dynamic auto/desirable DTP mode	Attacker's VLAN = trunk's native VLAN
Direction	Two-way (full trunk access)	One-way / blind only
Primary tool	Yersinia (DTP tab)	Yersinia (802.1Q tab) or Scapy
Validation	Wireshark shows trunk renegotiation + multi-VLAN tagged traffic	Wireshark shows traffic destined for the target VLAN's IP range
Kill switch (defense)	<code>switchport nonegotiate</code> + access mode on all edge ports	Move native VLAN off default/VLAN 1, explicitly tag it

GLOSSARY (garbled terms from the original lecture, decoded)

Heard as...	Actually means
"Villain" / "Vilian"	VLAN
"Trucking mode"	Trunking mode
"Cerny" / "Estonia" / "Senior" / "Hernia"	Yersinia (the attack tool)
"0.802 dot one Q" / "0.8021 Q"	802.1Q (the VLAN tagging standard)
"Sue do"	sudo
"Ashley work"	actually work

PRACTICE DRILL FOR YOU TO RUN

- 1 Build a small lab: 1 managed switch (or GNS3/EVE-NG virtual switch), 2 VLANs, 1 Kali attacker on VLAN A, 1 victim host on VLAN B.
- 2 Leave the switch on default DTP settings. Run the switch-spoofing steps in Section 02 and capture proof in Wireshark.
- 3 Reset the port to access mode + nonegotiate. Confirm the same attack now fails — this is the mitigation validation step clients expect.
- 4 Re-enable trunking but set an explicit, non-default native VLAN. Attempt double tagging and confirm it now fails too.
- 5 Write a one-paragraph finding + remediation as if this were a client report — this is the skill that actually gets tested in real engagements, not just running the tool.

SCOPE & ETHICS REMINDER

VLAN hopping techniques must only be performed in a lab you own or on an engagement covered by a signed statement of work / written authorization. Unauthorized VLAN hopping on a production network is unauthorized access to computer systems under most jurisdictions' computer-crime laws.